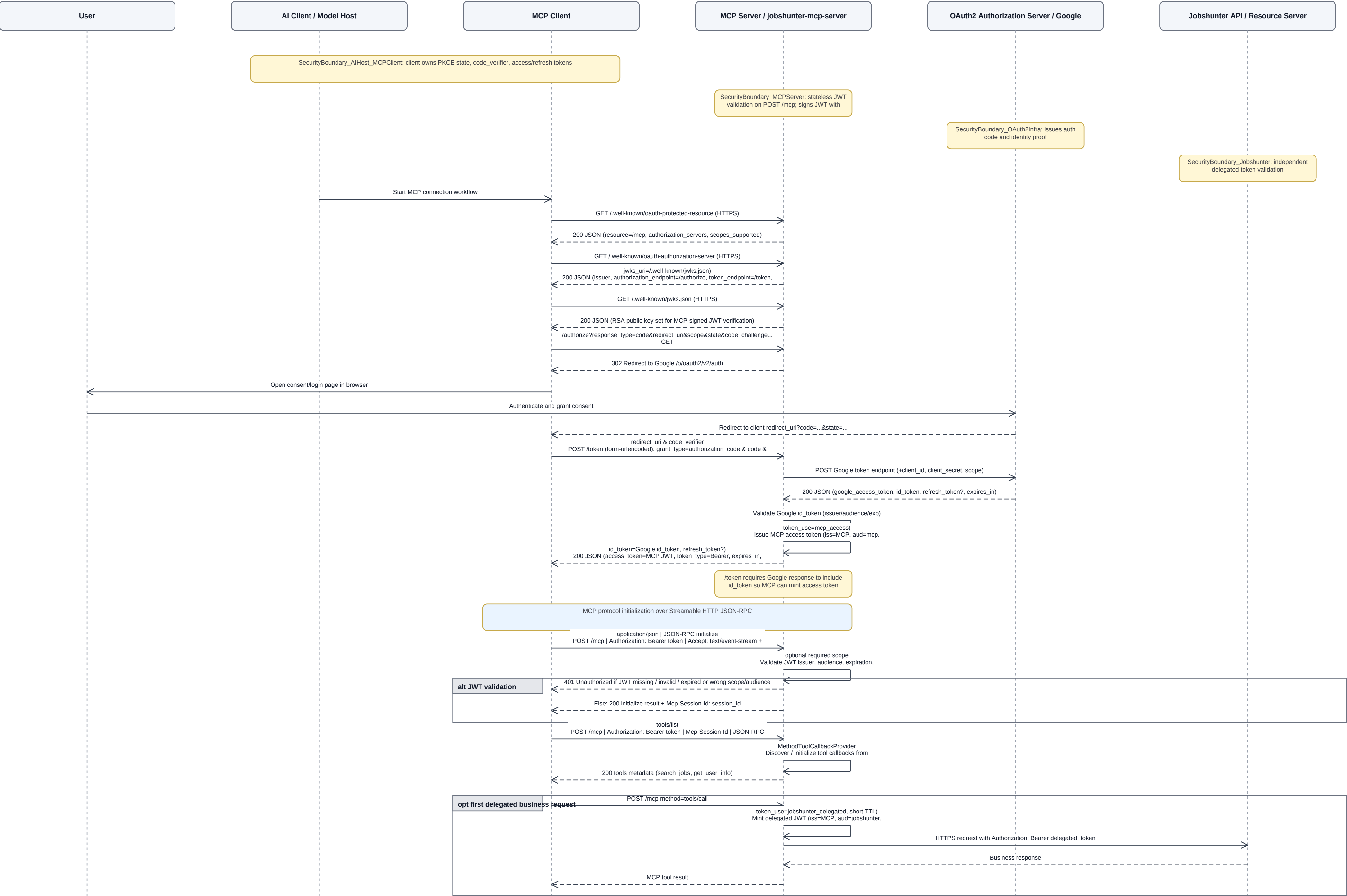


MCP Server Authentication and Initialization Flow

Sequence diagram reconstructed from the Mermaid source (.mmd)



Architecture and Security Notes

The MCP server acts as both a protected MCP endpoint and a first-party Authorization Server. It uses Google only as upstream identity proof, mints its own /mcp bearer token, and mints a separate delegated token for Jobshunter.

Important security and architecture considerations

- The server is stateless for OAuth sessions and token storage; the MCP client owns token lifecycle.
- `id_token` and `access_token` have distinct roles and are not rewritten.
- `Mcp-Session-Id` scopes protocol continuity, but authentication still depends on per-request bearer JWT.
- Delegated calls to Jobshunter use a dedicated short-lived JWT (`aud=jobshunter`) distinct from MCP bearer token.

Assumptions and unknowns (explicit)

- Spring AI internal JSON-RPC error envelope details are framework-managed and not fully specified by this repository.
- Refresh token behavior depends on upstream provider behavior and whether refreshed responses include `id_token`.
- Jobshunter-side JWT validation internals are external to this repository.

Appendix - Mermaid Source

The following appendix preserves the Mermaid source used to construct the sequence diagram.

```
sequenceDiagram
    autonumber
    participant User as User
    participant AIHost as AIClient_or_ModelHost
    participant McpClient as MCPClient
    participant McpServer as MCPServer_jobshunter_mcp_server
    participant Google as OAuth2AuthorizationServer_Google
    participant JobApi as JobshunterAPI_ResourceServer

    Note over AIHost,McpClient: SecurityBoundary_AIHost_MCPClient (client owns PKCE state, code_verifier, access/refresh tokens)
    Note over McpServer: SecurityBoundary_MCPServer (stateless JWT validation on POST /mcp, signs JWT with local RSA key)
    Note over Google: SecurityBoundary_OAuth2Infra (issues auth code and identity proof)
    Note over JobApi: SecurityBoundary_Jobshunter (independent delegated token validation)

    activate AIHost
    AIHost->>McpClient: Start MCP connection workflow
    deactivate AIHost

    activate McpClient
    McpClient->>McpServer: GET /.well-known/oauth-protected-resource (HTTPS)
    activate McpServer
    McpServer-->>McpClient: 200 JSON (resource=/mcp, authorization_servers, scopes_supported)
    deactivate McpServer

    McpClient->>McpServer: GET /.well-known/oauth-authorization-server (HTTPS)
    activate McpServer
    McpServer-->>McpClient: 200 JSON (issuer, authorization_endpoint=/authorize, token_endpoint=/token,
    jwks_uri=/.well-known/jwks.json)
    deactivate McpServer

    McpClient->>McpServer: GET /.well-known/jwks.json (HTTPS)
    activate McpServer
    McpServer-->>McpClient: 200 JSON (RSA public key set for MCP-signed JWT verification)
    deactivate McpServer

    McpClient->>McpServer: GET /authorize?response_type=code&redirect_uri&scope&state&code_challenge...
    activate McpServer
    McpServer-->>McpClient: 302 Redirect to Google /o/oauth2/v2/auth
    deactivate McpServer

    McpClient->>User: Open consent/login page in browser
    User->>Google: Authenticate and grant consent
    Google-->>McpClient: Redirect to client redirect_uri?code=...&state=...

    McpClient->>McpServer: POST /token
    (application/x-www-form-urlencoded)<br/>grant_type=authorization_code&code&redirect_uri&code_verifier
    activate McpServer
    McpServer->>Google: POST Google token endpoint (+client_id, client_secret, scope)
    activate Google
    Google-->>McpServer: 200 JSON (google_access_token, id_token, refresh_token?, expires_in)
    deactivate Google
    McpServer->>McpServer: Validate Google id_token (issuer/audience/exp)
    McpServer->>McpServer: Issue MCP access token (iss=MCP, aud=mcp, token_use=mcp_access)
    McpServer-->>McpClient: 200 JSON (access_token=MCP JWT, token_type=Bearer, expires_in, id_token=Google id_token,
    refresh_token?)
    deactivate McpServer

    Note over McpServer: /token requires Google response to include id_token so MCP can mint access token

    Note over McpClient,McpServer: MCP protocol initialization over Streamable HTTP JSON-RPC
    McpClient->>McpServer: POST /mcp<br/>Headers: Authorization Bearer token, Accept text-event-stream and
    application-json<br/>Body: JSON-RPC initialize
    activate McpServer
    McpServer->>McpServer: Validate JWT issuer, audience, expiration, optional required scope
    alt JWT missing/invalid/expired or unauthorized scope/audience
    McpServer-->>McpClient: 401 Unauthorized
    else JWT valid
    McpServer-->>McpClient: 200 initialize result + Header Mcp-Session-Id session_id
    end
    deactivate McpServer

    McpClient->>McpServer: POST /mcp<br/>Headers: Authorization Bearer token, Mcp-Session-Id session_id<br/>Body: JSON-RPC
    tools-list
    activate McpServer
    McpServer->>McpServer: Discover/initialize tool callbacks from MethodToolCallbackProvider
    McpServer-->>McpClient: 200 tools metadata (search_jobs, get_user_info)
    deactivate McpServer

    opt First delegated business request after initialization
    McpClient->>McpServer: POST /mcp method=tools/call
    activate McpServer
    McpServer->>McpServer: Mint delegated JWT (iss=MCP, aud=jobshunter, token_use=jobshunter_delegated, short TTL)
    McpServer->>JobApi: HTTPS request with Authorization Bearer delegated_token
    JobApi-->>McpServer: Business response
```

```
McpServer-->McpClient: MCP tool result
deactivate McpServer
end
deactivate McpClient
```